

RV32I Lab Simulator

Simulador educativo de RISC-V RV32I escrito en C++20.

1. Descripción general

Este proyecto implementa un simulador del ISA base RISC-V RV32I. El simulador mantiene el estado arquitectural básico de una CPU RISC-V: contador de programa PC, 32 registros enteros y memoria byte-addressable.

El programa permite cargar un binario crudo en memoria desde la dirección 0x00000000, ejecutar instrucciones paso a paso y consultar el estado de la CPU durante la ejecución.

Este proyecto no implementa un ensamblador. Los programas deben estar previamente compilados y exportados como binarios crudos.

2. Requisitos cubiertos

El simulador implementa los principales requisitos de la tarea:

- Carga de programas binarios crudos.
- Carga del programa desde la dirección 0x00000000.
- Memoria plana byte-addressable.
- Formato little-endian para lecturas y escrituras.
- Estado arquitectural con PC, 32 registros y memoria.
- Ejecución paso por paso.
- Ejecución de múltiples instrucciones con run N.
- Inspección de registros.
- Inspección de memoria.
- Detección práctica de bucles infinitos cuando el PC no cambia.
- Soporte para instrucciones RV32I base.
- Soporte básico para algunas llamadas ecall.

3. Lenguaje y herramientas

El simulador fue desarrollado en C++20.

Herramientas usadas:

- CLion
- CMake
- Ninja
- MSYS2 UCRT64
- GCC/G++

4. Estructura del proyecto

```
src/  
├─ main.cpp      Punto de entrada del programa  
├─ Memory.hpp    Interfaz de memoria  
├─ Memory.cpp    Implementación de memoria little-endian  
├─ Decode.hpp    Estructura de instrucción decodificada
```

— Decode.cpp	Decodificación de campos e inmediatos RISC-V
— Cpu.hpp	Estado y operaciones de la CPU
— Cpu.cpp	Ciclo fetch-decode-execute
— Shell.hpp	Interfaz de consola interactiva
— Shell.cpp	Implementación de comandos del usuario

5. Diseño del simulador

El simulador está dividido en tres componentes principales.

5.1. Memoria

La memoria está implementada como un arreglo de bytes mediante `std::vector<uint8_t>`.

Esto permite representar una memoria byte-addressable, tal como se requiere para RISC-V. Las operaciones de lectura y escritura de 16 y 32 bits se realizan manualmente en little-endian.

Por ejemplo, al escribir el valor `0x00000019` en memoria, los bytes quedan almacenados como:

```
19 00 00 00
```

5.2. Decodificador

El decodificador recibe una instrucción de 32 bits y extrae sus campos principales:

```
opcode
rd
funct3
rs1
rs2
funct7
```

Además, genera los inmediatos correspondientes a los formatos:

```
I-type
S-type
B-type
U-type
J-type
```

Los inmediatos con signo son extendidos usando una función de sign-extension.

5.3. CPU

La CPU mantiene:

```
PC
32 registros
memoria
estado halted/running
```

En cada instrucción se realiza el ciclo:

```
fetch -> decode -> execute -> update PC
```

También se fuerza que `x0` siempre valga cero, incluso si una instrucción intenta escribir en ese registro.

6. Instrucciones soportadas

El simulador soporta instrucciones del ISA base RV32I.

Loads

lb
lh
lw
lbu
lhu

Stores

sb
sh
sw

Operaciones inmediatas

addi
slli
slti
sltiu
xori
srli
srai
ori
andi

Operaciones entre registros

add
sub
sll
slt
sltu
xor
srl
sra
or
and

Inmediatos superiores

lui
auipc

Branches

beq
bne
blt
bge
bltu
bgeu

Salto

jal
jalr

System

ecall

7. Soporte básico para ecall

Se implementó soporte básico para algunas llamadas de sistema compatibles con el estilo usado en simuladores educativos como CPULator/SPIM.

El número de syscall se coloca en a7.

```
a7 = 1   imprimir entero
a7 = 4   imprimir string
a7 = 5   leer entero
a7 = 10  terminar programa
a7 = 11  imprimir carácter
a7 = 93  terminar programa
```

8. Comandos interactivos

El simulador tiene una consola interactiva con los siguientes comandos:

Comando	Descripción
help	Muestra los comandos disponibles
state	Muestra el PC y el estado de la CPU
step	Ejecuta una instrucción
run N	Ejecuta hasta N instrucciones
regs	Muestra todos los registros
reg NAME	Muestra un registro específico
mem ADDR COUNT	Muestra bytes de memoria
quit	Sale del simulador

Ejemplos:

```
state
step
reg a0
reg x10
mem 0x1000 32
run 100
quit
```

9. Compilación

En Windows usando MSYS2 UCRT64, CMake y Ninja:

```

$env:Path = "C:\msys64\ucrt64\bin;$env:Path"

$cmake = "C:\Program Files\JetBrains\CLion
2025.2.1\bin\cmake\win\x64\bin\cmake.exe"
$ninja = "C:\Program Files\JetBrains\CLion 2025.2.1\bin\ninja\win\x64\ninja.exe"

Remove-Item -Recurse -Force build -ErrorAction SilentlyContinue

& $cmake -S . -B build -G Ninja `
  -DCMAKE_MAKE_PROGRAM="$ninja" `
  -DCMAKE_C_COMPILER="C:/msys64/ucrt64/bin/gcc.exe" `
  -DCMAKE_CXX_COMPILER="C:/msys64/ucrt64/bin/g++.exe"

& $cmake --build build

```

10. Ejecución

Ejemplo:

```
.\build\rv32i_lab_sim.exe .\programs\mini_ecall.bin
```

Luego, dentro del simulador:

```

state
step
reg a0
quit

```

11. Programas de prueba

Se agregaron cuatro programas de prueba propios para validar el simulador.

11.1. mini_ecall.bin

Este programa prueba addi y ecall.

Comportamiento esperado:

```

a0 = 5
a7 = 10
Status = halted

```

Comandos sugeridos:

```

state
step
reg a0
step
reg a7
step
state
quit

```

11.2. store25.bin

Este programa calcula:

$10 + 15 = 25$

y guarda el resultado en memoria en la dirección 0x64.

Comportamiento esperado:

x3 = 0x00000019
mem[0x64] = 19 00 00 00

Comandos sugeridos:

```
run 20
reg x3
mem 0x64 4
state
quit
```

11.3. loop_sum.bin

Este programa calcula:

$1 + 2 + 3 + 4 + 5 = 15$

y guarda el resultado en memoria en la dirección 0x80.

Comportamiento esperado:

x1 = 0x0000000f
x2 = 0x00000006
mem[0x80] = 0f 00 00 00

Comandos sugeridos:

```
run 100
reg x1
reg x2
mem 0x80 4
state
quit
```

11.4. load_store.bin

Este programa guarda 42 en memoria, lo vuelve a cargar, suma 8 y guarda 50.

Comportamiento esperado:

x1 = 0x0000002a
x2 = 0x0000002a
x3 = 0x00000032
mem[0x90] = 2a 00 00 00 32 00 00 00

Comandos sugeridos:

```
run 50
reg x1
reg x2
reg x3
mem 0x90 8
state
quit
```

12. Decisiones de implementación

12.1. Memoria limitada

Aunque RISC-V usa direcciones de 32 bits, el simulador usa una memoria concreta de tamaño limitado para simplificar la implementación.

Por defecto se reserva:

64 KB

Esto es suficiente para los programas de prueba usados en el proyecto.

12.2. Detención por bucle infinito

Algunos programas de prueba terminan con una instrucción como:

```
jal x0, 0
```

Esto crea un bucle infinito. Para facilitar las pruebas, el comando run se detiene si detecta que el PC no cambió después de ejecutar una instrucción.

12.3. Instrucciones inválidas

Si el simulador encuentra un opcode, funct3 o funct7 inválido, reporta un error de ejecución y detiene la CPU.

13. Limitaciones

El simulador no implementa:

- Ensamblador.
- Carga directa de archivos ELF.
- Interfaz gráfica.
- GDB.
- Pipeline.
- Caché.
- Interrupciones.
- Sistema operativo real.

14. Conclusión

El simulador implementado permite ejecutar programas RV32I desde binarios crudos, inspeccionar el estado de la CPU y validar el comportamiento de instrucciones aritméticas, saltos, branches, loads, stores y llamadas básicas `ecall`.

El diseño modular facilita entender las partes principales de un simulador de ISA: memoria, decodificación y ejecución de instrucciones.